RESEARCH-ARTICLE

Accurate Differential Analysis using Record and Selective Replay

YUTA NAKAMURA, DePaul University, Chicago, IL, United States

XULU CHU, DePaul University, Chicago, IL, United States

IGNACIO LAGUNA, Lawrence Livermore National Laboratory, Livermore, CA, United States

TANU MALIK, DePaul University, Chicago, IL, United States

Open Access Support provided by:

Lawrence Livermore National Laboratory

DePaul University

Published: 23 June 2025

SSDBM 2025: 37th International Conference on Scalable Scientific Data Management
June 23 - 25, 2025
Columbus, USA

SSDBM '25: Proceedings of the 37th International Conference on Scalable Scientific Data Management (June 2025)
https://doi.org/10.1145/3733723.3733733
ISBN: 9798400714627

Accurate Differential Analysis using Record and Selective Replay

Yuta Nakamura
DePaul University
Computer Science
Chicago, IL, USA
ynakamu1@depaul.edu

Xulu Chu
DePaul University
Chicago, IL, USA
xchu3@depaul.edu

Ignacio Laguna
Lawrence Livermore National Laboratory
Computing
Livermore, CA, USA
ilaguna@llnl.gov

Tanu Malik
DePaul University
School of Computing
Chicago, IL, USA
tanu@cdm.depaul.edu

Abstract

To verify the reproducibility of an application, it is often necessary to execute it multiple times, each time with a different input, and evaluate whether the application outcome changes. Given the complexity of the software, any unexpected behavior in the outcome requires quick insight into application behavior. Efficient execution tracing is instrumental but traces must often be compared to attain that insight. Comparing execution traces is a significant challenge, especially for parallel MPI applications as tasks may exchange messages in a non-deterministic order. Current methods for comparing execution traces assume *same* input across application runs.

In this paper, we present a method to compare execution traces of a parallel MPI application that have two sources of non-determinism: a changed input and variations in message exchange order due to running the application at a different time. We show that to compare traces from such application runs, we need selective replay—a method to replay message exchanges only when the new execution, with a changed input, aligns with the recorded execution in terms of its execution path. We propose two methods for deciding selective replay, each of which vary in the amount of execution state maintained. Our results demonstrate that, without selective replay, multiple sources of non-determinism lead to numerous false positives in comparing two runs. While traditional record-and-replay methods can pinpoint the first location of divergence, they fail to identify subsequent points. Through selective replay, we uncover and explain all divergences and convergences, achieving a reduction in false positives by more than 50%.

CCS Concepts

• **Software and its engineering**; • **Computing methodologies** → **Parallel programming languages**;

SSDBM 2025, Columbus, OH, USA

ACM ISBN 979-8-4007-1462-7/25/06
https://doi.org/10.1145/3733723.3733733

Keywords

reproducible analysis, debugging MPI programs, record and replay, execution trace comparison, root cause analysis

ACM Reference Format:
Yuta Nakamura, Xulu Chu, Ignacio Laguna, and Tanu Malik. 2025. Accurate Differential Analysis using Record and Selective Replay. In *The International Conference on Scalable Scientific Data Management 2025 (SSDBM 2025), June 23–25, 2025, Columbus, OH, USA.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3733723.3733733

1 Introduction

Establishing the reproducibility of shared applications is often necessary for the purposes of rigor [31] and transferability of outcomes [23]. Reproducibility is often established in two or more steps [18]—the first step simply repeats the application to observe if outcomes match the stated or known outcomes. If the results match, further steps explore whether the application remains reproducible under different input conditions, such as changes in data, parameters, or environment. Recently, several methods [22, 29] have been explored that help to efficiently repeat applications. However, methods for efficiently reproducing applications under different input conditions are currently time-consuming or limited to sequential programs [21].

Consider a parallel scientific computing application running a galaxy formation simulation using Enzo's adaptive mesh refinement [25]. Two independent runs of this simulation yield a discrepancy. The first run with an input results in the successful formation of a specific galactic halo (halo 49). The second run (Run 2), with a different input, does not result in a halo, though per theory [30, 33] it should have formed. If the application is the same across runs, the input change in Run 2 could have influenced this outcome; the question remains: what is the underlying application behavior for this unexpected result—specifically the absence of the halo when the input was modified?

One common method to understand application behavior is by recording the execution traces [27]. However, as the above example shows the traces must also be compared to understand why the halo forms in one run and not in the other run. In parallel applications, comparing traces can be challenging since even if the input is the same, traces of two independent runs exhibit non-deterministic

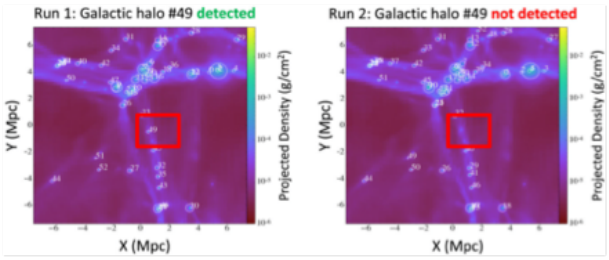

Figure 1: Discrepancy between two runs of galaxy formation simulation using the Enzo adaptive mesh refinement code [26] due to non-determinism. Documented by Stodden et al. in [25].

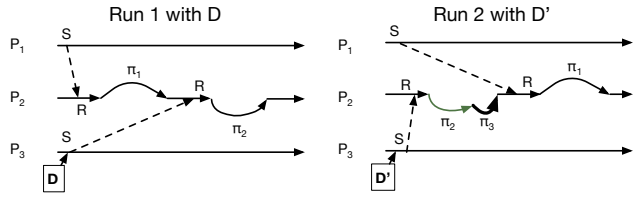

Figure 2: Execution path of the three ranks given input D at time t (left) and given input D' and time t' (right). Due to change in time, message orders are changed: In top P_1 sends message before P_3 and vice versa. Due to change in input D', P_2 traverses an extra subpath (π_3). `SelTraceReplay` aims to prevent the change of $\pi_1 \rightarrow \pi_2$ to $\pi_2 \rightarrow \pi_1$ and identifies the cause for π_3 as D'. Current tools cannot make this distinction.

order of execution, i.e., messages may arrive out of order due to communication non-determinism. Record and replay (RnR) methods record the order of message exchanges between parallel tasks, and during replay, sending processes follow the recorded order [14, 24]. While RnR controls order and makes two runs of an execution exactly the same, it assumes that the input is identical across both the recording run and the replay run. This assumption is true even for content-based record and replay methods [17, 38]. This raises the question of how to record and replay when the input is different.

Input changes often change a program's execution states. Thus record and replay must observe and determine if the state is same. If the execution state remains unchanged, we can control and replay messages reliably, preventing any out-of-order issues caused by communication non-determinism. When the states diverge, however, we can switch to following the natural order of execution based on the input changes, without controlling message exchanges. We call this approach selective replay, as opposed to the traditional exact replay currently used for trace comparisons with identical inputs.

We present `SelTraceReplay`, which enables selective replay of parallel MPI programs in which input changes may lead to inconsistent results. `SelTraceReplay` records order of execution of parallel tasks in a given run. In the subsequent run when specific inputs are changed, it replays recorded executions with selective replay. During selective replay, communication order is kept invariant and follows exactly the recorded execution as long as the execution state do not differ. We determine execution state in terms of both control flow path and the data exchange. If the execution control flow path differs, `SelTraceReplay` stops replaying but continues to observe the state of the process executions to precisely determine when the execution path matches with the recorded execution path so replay can resume. If the control flow path is same, same execution state is determined via message exchanges. In this way `SelTraceReplay` reports when the input changes affect replay and when it ceases to affect replay, thus reporting all locations which may cause execution path difference.

1.1 Our Contributions

Consider the example presented previously of comparing the output of two runs of a simulation of galaxy formation. Suppose three parallel tasks, P_1, P_2, and P_3 are involved in galaxy formation. Each of the parallel tasks, given an input, computes part of the galaxy formation but also shares with a controller process the current state of the galaxy formation, which processes it further to visualize it for the user. The order of message exchanges in Run1 is shown in Figure 2(left) where P_1 sent message to P_2 before P_3 and follows an execution path as shown. During Run2, the galaxy formation is experimented with input D'. In addition, since Run2 is at a different time, due to non-determinism, the order of messages exchanged is different with P_3 now communicating before P_1. The controller process P_2 responds to these two forms of non-determinism via a different execution path in which some part of the path is changed due to non-determinism and some due to changed input. Figure 2(right) shows the path changed due to non-determinism in gray and the path changed due to changed input via bold solid arrows.

Current approaches [24] cannot identify which part of the execution path differed due to communication non-determinism and which part of the path differed due to input change. Examining differences due to input change, and precisely locating functions in the execution path that are not behaving as expected, is often needed, especially when Run2 fails to produce a valid output. Our contributions aim to scope this search, and are the following:

- **A framework for selective replay.** We define selective replay in terms of execution paths and show that by making selective replay decision at specific locations of the execution path we can determine when the execution remains the same across two runs and when it differs.
- **Efficient methods for selective replay.** We design a method for selective replay based on path equivalence and order of message communication for resuming replay. Path equivalence is determined by an 'unrolling method' that uses the inferred nested loop structure to systematically mark loop iterations, i.e., start and end, and thus to easily compare two paths. We propose a first-come-first-out order of delivery of messages such that the two runs remain as "close" as possible despite the input differences.
- **An experimental prototype [7].** Our `SelTraceReplay` prototype uses LLVM [13] framework to obtain intermediate representation of parallel applications. The bitcode is combined with instrumented ReMPI [24] to record and replay traces based on selective replay decisions. We have experimented with several

```
A  int main(int argc, char **argv) {
A   int rank;
A  MPI_Init(&argc, &argv);MPI_Comm_rank(MPI_COMM_WORLD, &rank);
A   if (rank == 0) {
B       int i = 1;
B       while (i < argc) {
C           int val = std::stoi(argv[i]);
C           MPI_Send(&val, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
C           i++; }}
D   else { int i = 0;
D       while (i < argc - 1) {
E           int value;
E           MPI_Recv(&value, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
E  MPI_STATUS_IGNORE);
E           if (value % 2 == 0)
F           { std::cout << "even number" << std::endl; }
G           else { std::cout << "odd number" << std::endl; }
H            i++; }}
I   MPI_Finalize(); return 0; }
```

Figure 3: A source code of an example MPI program

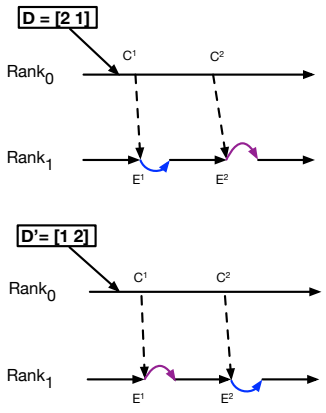

Figure 4: Simplified Execution Paths with D and D'

real MPI applications to show different kinds of divergences and convergences.

The rest of the paper is organized as follows: Section 2 defines execution path in a control-flow graph of a parallel application. Section 3 introduces and defines selective replay within a run of a parallel MPI application, and two sub-problems of determining when to resume replay and path equivalence. In Section 4 we describe our method of selective replay, including the solutions to the sub-problems. We present a detailed case study and experiment details on five real MPI applications in Section 5. We present other related work in Section 6, and conclude in Section 7.

2 Preliminaries

We define execution paths for an MPI program.

Figure 3 shows a simple MPI program P with a single function `main`. In this program, the central process (`Rank 0`) sends integer values, parsed from the command-line arguments, to the other processes (Ranks $\neq$ 0). The other processes receive these values and process them.

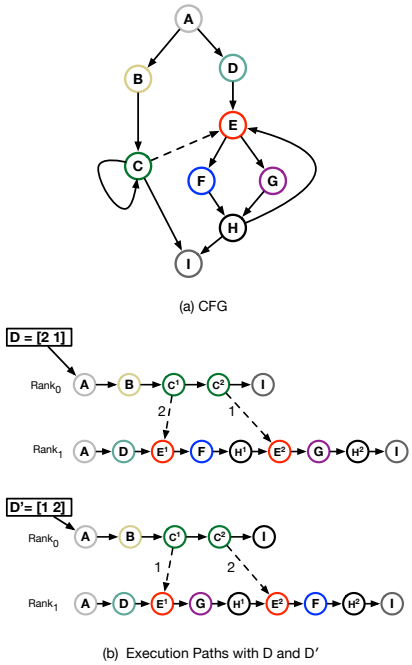

Figure 5: CFG and Execution Paths of the Program in Figure 3. Best visualized in color.

Figure 4 (top) shows the execution paths of the two ranks with input D = [2,1], and Figure 4 (bottom) shows the execution paths of the two ranks with input D' = [1,2]. In these execution paths, straight lines represent sequential instructions, and peaks and troughs represent control flow—a peak denoting the right branch and a trough denoting the left branch. The arrow between ranks denotes message exchange between a sender and receiver. In this program, message exchange between Rank_0 and Rank_1 respectively determines which branch is taken in the execution path.

We have made two assumptions in the above example:

- The source code in the example assumes a single `main` function, but typically, there are several functions within a program. In such cases, execution paths span interprocedural function calls. For clarity and ease of visualization, we focus on a single function in this presentation, though our method accounts for multiple functions (See Section 5 for more details).
- The messages follow a specific order due to the use of blocking MPI calls (`MPI_Recv/MPI_Send`) in the source code. Alternatively, if non-blocking MPI calls such as `MPI_Irecv/MPI_Isend` are used, then messages may potentially be exchanged out of order, leading to a different execution path as shown in Figure 5(b) bottom for input D'. While we use `MPI_Recv/MPI_Send` for the illustrative example, our solution is not limited to this call.

To compare execution paths across runs, the locations of the paths must be known. Generally, these locations are expressed in terms of basic blocks *i.e.*, a sequence of instructions that execute sequentially. For example, the MPI program P's source code in terms of control flow graph (CFG) is shown in Figure 5(a). In this graph, each node represents a basic block, and is given a unique

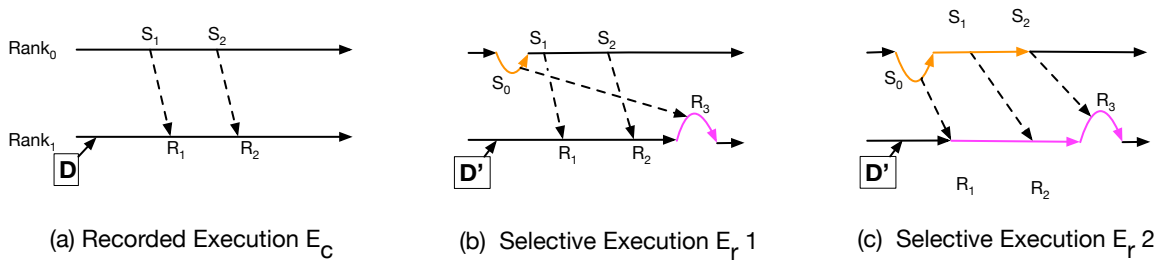

Figure 6: Should we receive the message the way Selective Communications 1 or 2? 1 seems more "similar" to recorded communications but the order of message exchanges is unlikely to happen without our toolset.

identifier $(A, \ldots, I)$. The solid edges between nodes indicate basic blocks that may be executed, depending on control flow conditions within the blocks. The dotted edges correspond to messages that could be exchanged between basic blocks of rank 0 process and any higher rank.

Execution of the program corresponds to a non-simple path in the control flow graph (CFG) from an entry point to the exit point of a function. In the example, the entry point of the function `main` is `MPI_Init()` and the exit is `MPI_Finalize()`. Figure 5(b (top)) shows the execution path in terms of basic block IDs taken by different ranks of P when it reads input D. A different control flow path is taken with input D' at Figure 5(b (bottom)).

When execution paths are simple (i.e., no loops) and do not exchange messages, the two paths can be compared (based on basic block ID locations) using edit-distance [32]. To compare paths that are not simple, *i.e.*, the path consists of loops, recently several methods have been proposed [19, 20]. These methods do not exchange messages and thus do not apply to parallel applications. In the next section, we determine how to compare execution paths that are non-simple and exchange messages.

3 The Selective Replay Problem

To effectively compare execution paths of a parallel program we must determine if messages are to be replayed or not. Intuitively, if the paths are diverged, `SelTraceReplay` should not replay the message; if not, it should. Our objective is to determine, in terms of basic block locations, *which* messages to replay.

We distinguish between a natural execution E of a program, its recording E_c, and its replay $R(E_c)$. A natural execution $E = P(t, D)$ at time t and with input D results in a partially ordered sequence of messages $(m_1, \ldots, m_n)$. The recording E_c consists of this partially ordered messages at specific basic block locations.

An execution at a different time $E = P(t', D)$ may result in a different sequence of partially ordered messages due to non-deterministic factors. The replay function R, replays the recording E_c at time t' and results in the same partial order sequence of messages $(m_1, \ldots, m_n)$ as at time t and thus no differences. This assumption requires P to behave deterministically under replay conditions. Thus, in Figure 5(b) top, if the execution is replayed at time t', messages may be exchanged as shown or they may potentially be exchanged between basic block locations C_1 and E_2 and C_2 and E_1. If the order is swapped, the content is swapped, and the path that will be taken is as shown in Figure 5(b) bottom.

Since our objective is to replay at t' with a different input D', we must determine which parts of E_c can be selectively replayed. For this, we define a selectively replayed execution $E_r = P(t', D')$ at time t' with a different input D' where $E_r = SR(E_c)$ where SR is the selective replay function. This function combines pure replay with natural execution at time t' and input D' by determining when replay is feasible.

To determine SR, we must solve the following two sub-problems:

- **Path Equivalence:** Determine if execution paths of recorded execution E_c and selectively replayed E_r are equivalent in terms of basic block locations. If locations are equivalent, then SR is equivalent to replay.
- **Ordering message exchange:** When path differ E_r cannot be replayed, and SR is false and must follow natural execution. However, during natural execution its messages must be ordered such that we can efficiently undertake replay when paths are the same.

Figure 6 illustrates why addressing both questions is crucial. We consider a program[1] in which both the sender and receiver exchange different numbers of messages under inputs D and D', respectively. Figure 6(a) shows the paths for $E_c = P(D, t)$, which exchanges two messages. When the input is changed to D', let the execution exchange three messages. Figure 6(b) and (c) show two possible ways of ordering message exchanges once path equivalence cannot be established. In Figure 6(b), all recorded messages are replayed exactly. However, if none of the messages are replayed, i.e., messages are delivered as they arrive, then E_r is as shown in Figure 6(c). As the figure shows, if we aim to adhere to recording locations, then Figure 6(b) is a better solution. However, if we keep the similar order of message exchanges, Figure 6(c) is more viable. Depending upon which E_r is considered, replay decisions change. Resulting execution path differences from the recorded execution path are shown in color/gray. Consequently, locations at which we define divergence and convergence change. In Figure 6(b), divergences are smaller, but messages have to be buffered. In Figure 6(c), they can be delivered as they arrive, but a longer divergence is reported. In the next sections, we describe how to determine when to selectively replay the messages among different ranks of a parallel MPI application.

[1] program listing is available here

4 Selective Order-Based Replay

In this section 4, we describe how selective order replay decisions are made. Replay decision occurs at every receive location in the execution path of a rank since a receiver may receive from multiple sender candidates. One may think it's possible to do it from senders too, but it's not because senders always have explicit destinations set, whereas receivers can do wildcard receives. The replay decision takes the recording E_c and the current location l as inputs, and determines for each receive location in any rank whether that location should be replayed. To present selective order replay, we first describe how messages are ordered, assuming the execution path has unique locations. We then describe a method for determining unique locations in a path so path equivalence is established (subsection 4.2).

4.1 Ordering Message Exchange

A straightforward approach to message ordering involves allowing each rank in E_r to "locally" decide whether to replay a message. In this approach, a receive call on any rank checks whether its execution path up to that point matches the corresponding recorded rank's path. Path equivalence, as described in Section 4.2, is used to determine this match. If the paths are equivalent, the message is replayed using the source specified in the recorded location. Otherwise, the natural execution of `MPI_Recv` proceeds, allowing the rank to receive the message from whichever source.

For non-blocking operations (e.g., `MPI_Irecv`), replay can be enforced locally similarly. However, in the case of `MPI_Irecv`, it is also necessary to "look ahead" in the recorded trace to identify the message source, using MPI calls from the wait or test families (e.g., `MPI_Wait`, `MPI_Test`, `MPI_Testsome`, `MPI_Waitall`). Alternatively, if messages include tag information, the receiver can perform tag equivalence checks at its location. Algorithm 1 outlines this straightforward local approach in which unique location l is looked up if it exists in the recording E_c and if it does, then replay from that source (Line 5). For the sake of simplicity, we do not account for *tag* in Algorithm 1 (Algorithm 2 too), and we assume that they are `MPI_ANY_TAG` both for senders and receivers.

This method ensures that replay halts when the receive location l of a rank is not found in the recorded execution and resumes when the receiver's execution path aligns with the recorded path. Thus, replay is never forced when the receiver's execution path diverges. However, if the sender in E_r has diverged and does not intend to send a message, while the receiver remains aligned, local selective replay will still wait for the expected message from the sender. Since the receiver only enforces replay by observing its own execution path, the message will never arrive, causing it to stall.

This scenario is illustrated in Figure 7, where the first two messages are replayed locally, but the third message causes a stall because the sender has diverged while the receiver continues to expect the message due to the local replay mechanism.

To avoid such 'stalling,' a more effective approach is for the sender to include its execution path location as part of the message, allowing the receiver to explicitly verify it. Message ordering is achieved by piggybacking the sender's execution path location whenever a rank sends a message. Additionally, when a receiver needs to wait, the counter of messages sent per rank is also piggybacked. This piggybacking occurs during both the recording and replay phases, as the matching performed during the replay phase relies on the information captured during the recording phase.

Algorithm 1: MPI_Recv

input : buf, cnt, $datatype$, src, tag, $comm$, $status$
output: return value of `PMPI_Recv`

```
1  Function MPI_Recv(buf, cnt, datatype, src, tag, comm,
     status):
2      l ← self.getRecvLocation()
3      if l ∈ E_c.getAllRecvLocations() and
4      src = MPI_ANY_SOURCE then
5          src ← E_c.getRecordedSourceRank(l)
6          l.replay ← true
7      else
8          l.replay ← false
9      rv ← PMPI_Recv(buf, cnt, datatype, src, tag, comm,
         status)
10     return rv
```

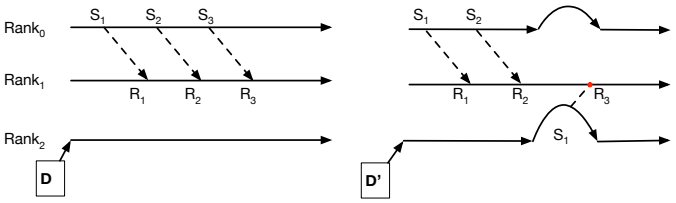

Figure 7: When local alignment fails

Messages may still arrive out of order (due to delays) or may not arrive at all due to network delays. To handle message delays and out-of-order arrival, we use a buffer to store incoming messages. During `MPI_Init`, a dedicated thread is created to listen for incoming messages and populate buffers maintained for each rank. When the target program invokes `MPI_Recv` (illustrated as R_1, R_2, R_3 in Figure 8), the system checks the buffer for the required message instead of directly calling the PMPI interfaces. We consider the case of message non-arrival later.

Algorithm 2 outlines the decision process for an `MPI_Recv` call when messages are delayed and arrive out of order. The algorithm ensures the correctness of the target program's execution by avoiding the replay of messages with mismatched tags at a receive or send location, and by preventing the same send message from being matched to multiple receives. The core idea is to continue to replay if there exists a message from the recorded sender, but also observe if the message is delayed or the sender diverged and sent some extra messages. If there is a delay or extra messages, then do not replay, even if there is message sent from recorded send location, but deliver the messages from the buffer first-come first-serve. For example, given the two choices to replay messages in Figure 6(b) and (c), Algorithm 2 will choose replay order that resembles (c) instead of (b) since this message exchange resembles closely to the

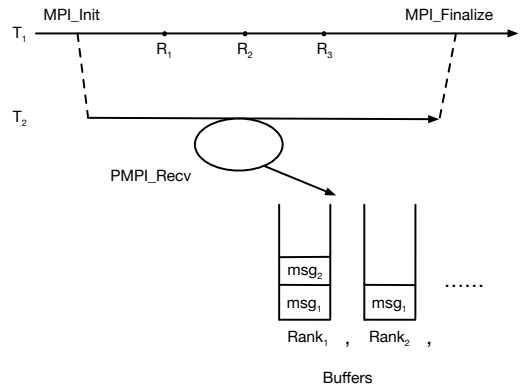

Figure 8: Message Listening and Buffering

recorded execution in Figure 6(a). Note, if the recorded execution had out-of-order message exchanges, then it will record the out-of-order message exchanges, and replay in that order as long as the path is the same.

In Algorithm 2, we obtain the current location of the receive call (at line 2) *i.e.*, l, and check for local alignment at line 3. If the local alignment fails, the algorithm retrieves the earliest appropriate send messages from line 23 to line 28. When local alignment is successful, the algorithm finds the corresponding recorded send location k for l from E_c (at line 4). After knowing the potential send location from the recording, the algorithm looks into its current buffer of messages to determine which send location message to match to this receive location. To do so, it sorts messages in the buffer (line 5 to line 10). Specifically, all messages are sorted based on the timestamp of arrival at the buffer. However, if a message is delayed *i.e.,* it is an out-of-order message then the message timestamp is swapped so as to always consider a first-come first-serve order.

After the messages are sorted in the buffer, the algorithm determines which message (and its corresponding location) from the buffer to deliver to this specific receive call, based on the following decision order:

- Conditions at Line 15. If there exists any message from a sender's rank that is not recorded in E_c or there exists an outstanding message (*i.e.*, whose receive location was already served with a previous message), then consider such a message to match with this receive location (Line 14 and 15). This condition occurs when the sender sends messages in a diverged state, or some outstanding messages from the sender that have not been matched at the receiver rank. This condition is to satisfy the first-come, first-serve order.
- Conditions at Line 17. Replay the message obtained from the recorded send location (line 18). This is the case where recording has the out-of-order message recorded, and no divergence has happened. In that case, we wish to replay according to the recording.

If no such message exists, continue to the next message. Whenever a message from the buffer is matched with the current receive location. it is evicted from the buffer.

Algorithm 2: MPI_Recv for nonlocal alignment

input : buf, cnt, $datatype$, src, tag, $comm$, $status$
output: return value of MPI_Recv

```
1  Function MPI_Recv(buf, cnt, datatype, src, tag, comm,
     status):
2      l ← self.getRecvLocation()
3      if l ∈ E_c.getAllRecvLocations() then
4          k ← E_c.getMatchingSendLocation(l)
5          if src = MPI_ANY_SOURCE then
6              sortedMsgs ← buffer.sortMsgsBySentTS()
7          else
8              sortedMsgs ←
                buffer.sortMsgsBySentTSPerRank(src)
9          if k ∉ sortedMsgs.getAllSendLocations() then
10             goto line 24
11         foreach m in sortedMsgs do
12             k_tmp ← m.getSendLocation()
13             l_tmp ← E_c.getMatchingRecvLocation(k_tmp)
14             if k_tmp ∉ E_c.getAllSendLocations() or
15             l_tmp < l then
16                 l.replay ← false
17             else if k_tmp = k then
18                 l.replay ← true
19             else
20                 continue
21             buffer.evict(m)
22             break
23     else
24         if src = MPI_ANY_SOURCE then
25             m ←buffer.getEarliest()
26         else
27             m ←buffer.getEarliestFromRank(src)
28         buffer.evict(m)
29     buf ← m.buf
30     status ← m.status
31     rv ← m.rv
32     return rv
```

Finally, if a specific message is delayed forever, or may not arrive, Algorithm 2 may still stall the receiver since it assumes a message from recorded rank will necessarily arrive. Clearly, waiting forever is not feasible. The system stops waiting and the receive location gets any next message if it observes that the buffer consists of messages that are no earlier than the recorded send location in location order, and all the messages from locations previous to the recorded location have been matched (through the use of send counters that are piggybacked).

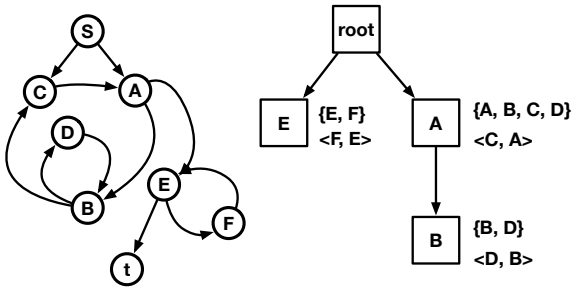

Figure 9: A CFG and its loop tree

4.2 Determining Unique Locations for Path Equivalence

The selective replay decision crucially depends on unique locations for each basic block that has the receive call. In general, determining unique locations in execution is challenging since iterative basic block traversals, which are common in MPI applications, must be distinguished from each other. We rely on an "unrolled" control-flow graph (CFG) of the program to determine the unique locations of each basic block. We describe this procedure briefly here with more details in [19]. The CFG corresponds to the LLVM-IR representation which for any general program is a directed graph (Figure 5(a) in Section 2). The unrolling relies on first identifying strongly connected components (SCC) over the directed graph of the CFG. The resulting SCCs are organized in a loop tree, which organizes SCCs in a hierarchical structure of nested and disjoint loops. It also marks entry points of loops and their back-edges. In case the loops are nested, we omit an arbitrary entry node from the SCC and find other SCCs in the original SCC.

Figure 9 illustrates the identification of SCCs and the creation of loop trees. A CFG corresponding to LLVM-IR is shown in Figure 9 (left), and its corresponding loop tree on the right. In this loop tree, each node is an SCC, specified by elements of the SCC in curly brackets, the entry node of the SCC as the node identifier, and the back-edge in angled brackets.

Given an execution path as shown at the top of Figure 10, the tree can be used to "unroll" the path into a sequence of SCCs each with distinct identifiers as follows (with purple vertices and edges at Step 3 of Figure 10):

(1) Omit the back-edges from the CFG (Edge (C, A)).
(2) Create loop iterations (vertices and edges within the loop minus the back edges) incrementally based on number of loops witnessed in the path. The incremental creation is since the execution path in the reproduced execution appears one basic block at a time unlike the entire path shown in the example.
(3) Create edges from the sources of the back edges to the destinations of the back edges in the next iteration of the loop (red dotted edges), and edges that exit from the loop for each iteration (blue dotted edges).

The result of the three steps is that each location in a path with ambiguous basic block locations is uniquely identified. To illustrate how this procedure unrolls, we further consider a CFG with nested loops, unnatural loops, and self loops (Figure 11 left). We

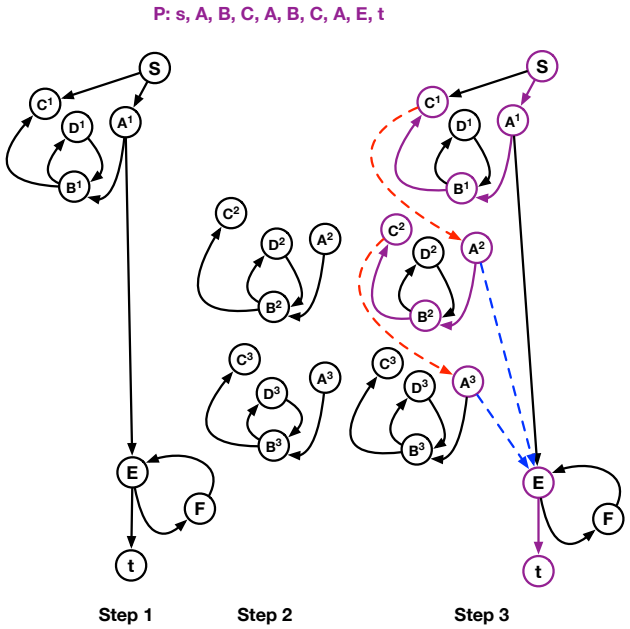

Figure 10: Unrolling the loop $\{A, B, C, D\}$

also consider a loop tree corresponding to SCCs identified in this CFG (Figure 11 right). Finally we consider a recorded trace over this CFG as shown in Figure 12. Whenever a vertex node corresponding to an entry of the loop is seen in the recorded trace, a virtual node is created which represents that loop. We denote this virtual node as (x) where x represents the entry point of the loop. Subscripts of the virtual nodes represent the number of iterations of the virtual node (*i.e.*, ith iteration of loop with an entry node x with $(x)_i$). In Figure 12 we show how each of the ambiguous locations in the recorded trace are given a unique location based on virtual node delineation. For example, $(A)_1$ has a subpath $(A, (B)_1, (B)_2, D)$ and each (B)s has its own loop iteration (for $(B)_1$, it is $(B, (C)_1, (C)_2, E)$) in Figure 12. We did not specify the loop iterations with entry node C because it is a self-loop and is always (C). As the example shows, the unrolling of loops can be accomplished at runtime of the execution. The algorithm for creating virtual nodes is linear, unlike unrolling the given CFGs.

Note once the unique locations are known the sorting function in Algorithm 2 is well-defined, and establishing execution path equivalence in local vs non-local selective order-based replay decisions is possible.

5 Experiments

We evaluate how accurate and efficient differential analysis is using `SelTraceReplay`. Our open-source implementation is available via [7]. *In particular, we determine if using* `SelTraceReplay` *leads to more precise identification of where program execution paths differ, and if so then at what recording and replay overhead.* We show that on average with 2.5 times the overhead we increase accuracy by 50%.

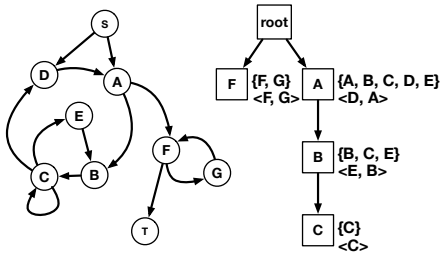

Figure 11: A CFG with nested loops, unnatural loops and self loop (left). Its corresponding loop tree (right)

Trace: s, A, B, C, C, E, B, C, D, A, B, C, D, A, F, G, F, t

Trace : s, $(A)_1$, $(A)_2$, $(A)_3$, $(F)_1$, $(F)_2$, t

$(A)_1$: A, $(B)_1$, $(B)_2$, D

$(B)_1$: B, $(C)_1$, $(C)_2$, E

$(B)_2$: B, $(C)_1$

$(A)_2$: A, $(B)_1$, D

$(B)_1$: B, $(C)_1$

$(A)_3$: A

$(F)_1$: F, G

$(F)_2$: F

Figure 12: Unrolling a Trace

To evaluate the accuracy, we compare `SelTraceReplay` with two other known methods of differential analysis **Diff** and **Greedy**. To evaluate the efficiency, we compare with these methods in addition to comparing with **ReMPI**, which records and replay but cannot take different inputs for replay (and therefore was omitted from accuracy results). All three methods are explained below.

Baselines. Diff and **Greedy**, only find points of divergence and convergence but do not control any non-determinism. **Diff** is an edit-distance based method [32] available as part of Linux utilities to compare to execution paths of two runs. In our case, it is the first run is the recorded execution path E_c and the second run is the recording of the reproduced run without any selective replay. When 'diff' compares, it considers neither calling context (*i.e.*, call stack information) nor loop iteration information. Thus, it also has no unique location information. **Greedy** is a greedy differential analysis method [19] that compares the two runs if the calling context of the execution and the number of loop iterations are the same. Thus, greedy has unique location information for each basic block, but the method does not employ any form of selective replay. ReMPI is an order-based record and replay method [24], which controls non-deterministic delivery of messages when an execution is run multiple times with the same input. Finally, we use `SelTraceReplay` in **Local** and **Non-local** modes. Local represents Algorithm 1 and **Non-local** represents Algorithm 2.

We also present a baseline experiment using foundational models. Given the wide popularity of foundational models, we were curious if, given entire code repositories, foundational models can precisely identify function locations where execution paths differed.

Dataset of Parallel Programs. Many MPI-based parallel programs implemented in C/C++, allowing us to obtain their control-flow graphs (CFGs) using the LLVM toolset [13]. Since MPI does not encourage shared-memory access, we only investigate point-to-point (P2P) MPI calls.

We experimented with several MPI programs specified in the 100-benchmark MP dataset [12]. This is the most comprehensive dataset of MPI programs constructed so far. We tested 30 out of 100 programs given the available time resources. 25 of them did not exhibit communication non-determinism. We chose five prominent programs that exhibited communication non-determinism as different runs produced different orders of message exchanges. Table 1 shows the five modules. **mini mcb** is a example program used by ReMPI. AMG 2013 is a parallel algebraic multigrid solver for linear systems arising from problems on unstructured grids. It has been derived directly from the BoomerAMG solver in the **Hypre** library, a large linear solver library that is being developed in the Center for Applied Scientific Computing (CASC) at LLNL. SMG, PCG, and PFMG are different independent modules in the Hypre library[16]. Since we are experimenting accuracy of differences in complex and large MPI codes, and comparing with known differences, a primary reason for choosing these applications was human familiarity with them since they were also developed at one of the author's institution.

Table 1 also shows the executions commands on the five modules. Changed variables are in bold. Since we are always comparing two executions we show results across the two most prominent variable changes that made the most effect.

Table 1: Recording and Reproduce Commands for Experiment Modules

Modules	Commands
mini mcb	`./rempi_test_mini_mcb 10 1 3` `./rempi_test_mini_mcb` **`11`** `1 3`
AMG 2013	`./amg2013 -P 2 2 2 -n 10 10 10` `./amg2013 -P 2 2 2 -n 10 10 10` **`-solver 1`**
SMG	`./ex3 -n 5 -solver 1 -v 1 1` `./ex3 -n 5 -solver 1 -v` **`2 2`**
PCG	`./ex1` `./ex1` **`-vis`**
PFMG	`./ex7 -n 5 -solver 1 -K 3 -B 0 -C 1 -U0 2 -F 4` `./ex7 -n 5 -solver 1` **`-K 4`** `-B 0` **`-C 2`** `-U0 2 -F 4`

To present our results, we first present a case study, and based on the thorough analysis of the case study, we then present accuracy and efficiency results over the five MPI-based parallel programs.

5.1 Case Study: PFMG (hypre)

HYPRE is a library of high-performance preconditioners and solvers for solving large and sparse linear system of equations. It is especially useful for solving partial differential equations. It is implemented in C and uses MPI for parallelization. Parallel Fine/Coarse MultiGrid (PFMG) is a solver that does more optimized computations for regular and structured grids (*i.e.* a domain where the

```
1 hypre_ExchangeDataList() {
2     initialize_communication();
3     initiate_nonblocking_sends();
4
5     received_count = 0;
6     total_needed = compute_total_needed();
7
8     while (received_count < total_needed) {
9         (flag, source) = MPI_Iprobe(MPI_ANY_SOURCE);
10        if (flag) {
11            data = MPI_Recv(source);
12            process_data(data);
13            received_count += 1;
14        }
15    }
16
17    finalize_exchange();
18 }
```

Figure 13: Simplified pseudocode for `hypre_ExchangeDataList`

problem is solved), as opposed to Algebraic Multigrid (AMG), which solves the equation in more general settings.

We changed 2 input parameter values, diffusivity and discontinuous coefficients, in PFMG resulting in two execution paths. During recording we experiment with diffusivity of 3 and discontinuous coefficient of 2. During reproduction, we increase diffusivity coefficient from 3 to 4, and the discontinuous coefficient from 1 to 2. The diffusivity coefficient represents how fast the heat or temperatures disperse as the coefficients increase. The discontinuous coefficient represents the gradients among the values of the grid.

These changes should cause points of divergence to appear. We discovered via manual inspection of source code that the function `hypre_ExchangeDataList` (Figure 13) has multiple sources of non-determinism. This function exchanges data among ranks in a non-deterministic fashion. In Figure 13 `MPI_Iprobe` at line 9, looks for messages from any source (`MPI_ANY_SOURCE`). If a message was received from an arbitrary source, it gets the message with `MPI_Recv` (line 11) and sends the response back. If the messages have not arrived, it loops to `MPI_Iprobe` until `received_count` increments up to `total_needed` (line 8). `MPI_Iprobe` may also fail to fetch messages if the messages did not arrive promptly.

We aimed to determine if this function could be accurately and efficiently obtained by changing input parameters. Table 2 shows the points of divergence for greedy method and local and nonlocal modes of `SelTraceReplay`.

Since the greedy method does not control non-determinism due to message exchanges, the greedy method finds five more points of divergence at `hypre_ExchangeDataList`, and non-determinism causes these (*i.e.*, they are false positives). All the other points of divergence turned out to be the same for both methods and correct when checked manually.

5.2 Research Question 1: Accuracy

In general determining accuracy can be challenging since for complex code bases there is no absolute ground-truth available. Given our familiarity with these code bases, we began with some ground truths. Table 3 shows the functions with known points of divergence, and we classified the cause either due to communication order (first column) or input differences (second column). Notice

Table 2: Comparison of Points of Divergence

Greedy	Nonlocal Alignment
C:0	
hypre_DataExchangeList:32	
hypre_DataExchangeList:40	C:0
hypre_DataExchangeList:43	hypre_PFMGSolve:9
hypre_DataExchangeList:44	hypre_PFMGSolve:16
hypre_DataExchangeList:48	K:0
hypre_PFMGSolve:9	
hypre_PFMGSolve:16	
K:0	

Table 3: Functions with Points of Divergences and Causes

Modules	Communication Order	Inputs
mini mcb	`neighbor_communication`	`main`

AMG 2013	`hypre_DataExchangeList`
	`hypre_FillResponseIJ-DetermineSendProcs`
	`hypre_PCGSetup`
	`hypre_qsort2i`
	`HYPRE_SStruct-GraphAssemble`
		`hypre_PCGSolve`
		`main`

SMG	`hypre_DataExchangeList`	`hypre_SMGRelax` `hypre_SMGSolve`
PCG	`hypre_DataExchangeList`	`main`

PFMG	`hypre_DataExchangeList`	`hypre_PFMGSolve`
		`C`
		`K`

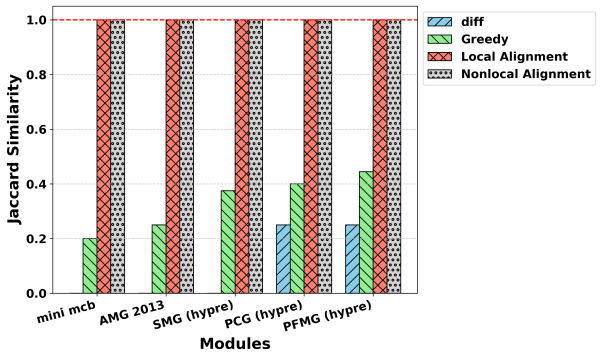

Figure 14: Accuracy of Points of Divergence Per Module

that AMG2013 [2] and the solutions in Hypre [16] share `hypre_DataExchangeList` as a function that shows points of divergence due to communication orders. The modules share many codes and this function turns out to show the non-determinism for many modules. These ground truths were independently examined and verified by one of the co-authors.

To measure accuracy, we compute the Jaccard similarity [10] in which the numerator is the intersection of which functions were common to ground truth and denominator is the union of all functions in ground truth at `SelTraceReplay` Figure 14 shows the Jaccard similarity [10] result.

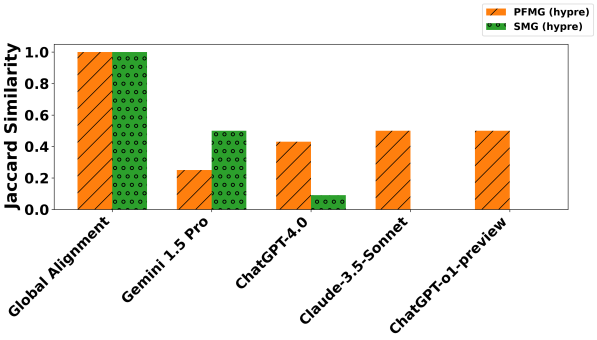

Figure 15: Accuracy of LLM models

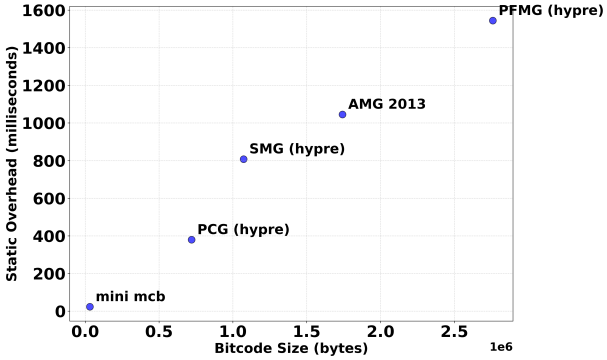

Figure 16: Relationship with Bitcode Size and Compile Time Overhead

Linux 'diff' command generally showed wrong points of divergence, which was expected without the calling context and the considerations of loop iterations. However, greedy method (with loop consideration) did not do well in terms of accuracy mainly because of the false positive results it showed. Greedy method, which does not enforce any communication order, detects the points of divergence due to such cases, and that severely compromises accuracy. However, there were no false negatives with the greedy method i.e., it did find all the points of divergence which `SelTraceReplay` also found. Overall, our tool finds accurate points of divergence more than any other tool that's available as a status quo.

As illustrated in Figure 15, we also compared the accuracy with large online language models. The accuracy is represented by Jaccard similarly to before. The experiment was done over the module only in SMG and PFMG because we hypothesized that the program was complicated enough to have difficulty finding the correct points of divergence. The commands are the same as the other experiments (*i.e.*, shown in Table 1).

We combined methods called role-prompting [6] and chain-of-thought prompting [35] in this experiment, which is known to improve its accuracy. In role-prompting, users feed the role to the LLM, and in chain-of-thought prompting, instead of feeding instructions at once, we used a technique to feed the model step-by-step instructions. We fed the model the role of a C programming and MPI debugging expert. We also fed the source code to the LLM, exact commands shown in Table 1, and environments (*e.g.*, ran on Ubuntu 20.04, uses open MPI library with version 4.0.3) into each LLM tool available today (shown in Figure 15 as abscissa) as inputs. Lastly, we told the models to find the locations affected via argument changes at which part of the code in line numbers as an output. Gemini 1.5 Pro and ChatGPT-4o find all the points of divergence with some false positives, while the rest do not find all the points of divergence (*i.e.*, false negatives exist). From Figure 15, we conclude that, though LLMs give some hints over points of divergence, they would significantly lack accuracy compared to our tool.

5.3 Research Question 2: Overhead

In this experiment, our objective is to determine the overhead of statically modifying the source code using the LLVM toolset, obtaining the recording, and selectively replaying the modules. Notice that statically modifying the source code is a one-time cost, unlike recording and selectively replaying, which are recurring as one executes the target applications.

Statically modifying the code could cause a significant overhead, but since it only happens per executable file, one could be much more lenient than runtime overheads. Figure 16 shows the static overhead per module in milliseconds on the y-axis and the original bitcode size to modify on the x-axis. We need to modify the bitcode so that each basic block traversal is known to users for comparison of the executions. Moreover, we need to analyze the CFGs of each function in the bitcode to find loops and create loop trees per function. As one can see from Figure 16, these tasks would take longer to process as the size of bitcodes becomes larger, and the figure cleanly represents such a trend, but in absolute numbers, even when the bitcode size is highest, static overhead is barely 1.6s.

Figure 17 shows the recording overhead compared to the vanilla execution time (termed raw runtime). The recording overhead of `SelTraceReplay` in local and non-local mode is, in general, slightly more than the greedy method, since the greedy method only records the basic block executions traversed, where as our method piggybacks and records more details about the send locations.

In local and non-local mode, we distinguish overheads between two cases, one when we record every basic block location versus recording when send/receive messages are exchanged. Recording more basic blocks allows us to be more precise in terms of divergence and convergence locations, while recording at message sends/receives allows us to be only as precise up to the basic block locations which send messages. Thus in the latter case, a basic block that did not send/receive message will not be a point of divergence. When inputs change often times there are divergences that take different execution paths unrelated to messages/send receives and such divergences can be ignored. These overheads are shown stacked in different patterns in Figure 17. Clearly the maximum overhead for AMG 2013 and SMG and about 3X in case we only record send/receive message locations and about 7X when we record every basic block location.

The primary cause of large overhead is due to unrolling of loops and determining the unique location identifier of the basic blocks at every appropriate MPI call (*e.g.*, `MPI_Recv`). This overhead increases as more memory is needed as the number of basic blocks

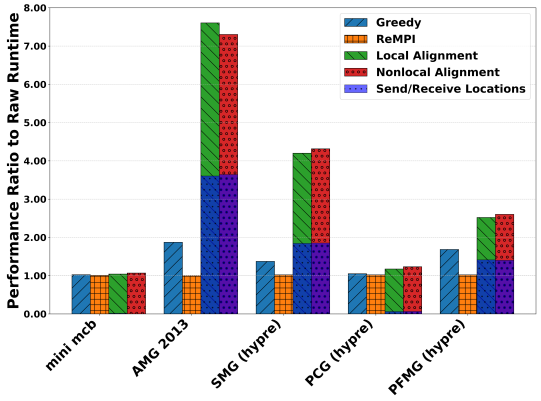

Figure 17: Recording overhead compared with raw runtime

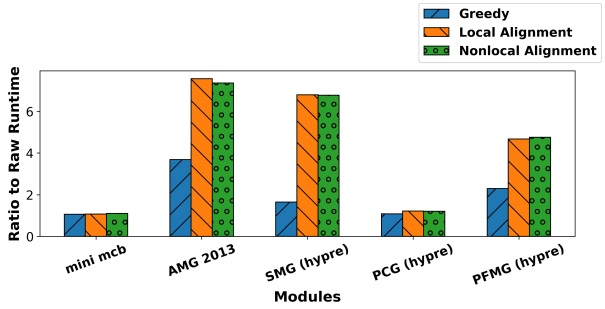

Figure 18: Selective Replay overhead compared with raw runtime

is traversed and analyzed. While we think paying an overhead of 3-7x is not high given that debugging large programs can be quite time-consuming, sometimes even taking months [27], and precisely identifying locations of divergence in an automated fashion can be helpful. We also do believe that this overhead can be further reduced with more memory optimizations, such as by using encoding and compression.

Given a recording, we compute overheads during the selective replay execution. Note that selective replay means combining natural execution with some replay. The overhead is similar to recording for the greedy method, local alignment, and nonlocal alignment, as shown in Figure 18. The primary overhead is due to determining the basic block locations to be unique, as is the case with recording overhead. The greedy method does not need this procedure for recording because we do not need to decide whether to replay with path equivalence, hence the tiny overhead during the recording phase. We realized that sometimes the unexpected points of divergence create page faults, and that further adds to the overhead.

To our surprise, there were few overhead differences between local and nonlocal alignment for both record and selective replay phases. This suggests that the overhead of piggybacking, recording, and checking send locations is not the bottleneck.

Discussion. We have demonstrated techniques for differential analysis of execution traces under non-determinism, but several limitations remain, presenting future opportunities. A formal proof of our algorithm's correctness is pending and will be detailed in future work. Performance overhead during record and replay remains a concern, though we introduced three key optimizations: virtual nodes for loop iteration tracking, location matching for receive alignment, and piggybacking send counters to prevent deadlocks. These optimizations already improve trace analysis efficiency compared to manual methods. Looking ahead, we aim to apply our tool to aid machine learning debugging by feeding trace insights into LLMs, enhancing their ability to correct or explain the code they generate.

6 Related Work

While we applied many of the concepts discussed in the papers reviewed in this section, our method of selectively replaying communication in the same order as the original execution is novel. We demonstrate this originality through comparisons with prior work in differential analysis, as well as the sources of ideas that informed our implementation (*e.g.*, uniquely identifying nodes and enforcing communication order as described in this paper).

Differential Analysis. Differential analysis has been extensively studied for various purposes, such as workflow analysis [3, 21] and security breach detection [8, 11]. For workflow analysis, [3] examines scenarios where one input yields the desired outcome while another does not. By analyzing execution paths corresponding to these inputs and identifying differences with software tools described in [3], the root causes of desired outputs are uncovered. Similarly, [21] compares faulty and correct executions to pinpoint the root causes of faulty behavior via differentiation.

In the context of security breach analysis, methods such as [8, 11] rely on combining multiple execution paths into a single control flow graph (CFG). Execution paths within the graph are considered "normal," while paths that deviate from the graph trigger alerts for abnormal behavior. However, none of these methods explicitly address the nondeterminism introduced by communication order, which is critical for differential analysis of distributed systems.

Multithreaded Differential Analysis and Trace Tools. Tools such as Structural Clustering and DiffTrace [28, 34] perform differential analysis on multithreaded programs by leveraging formal concept analysis [37] and similarity indices to detect unexpected executions. Similarly, tools like Vampir [5] and the Intel Trace Analyzer [9] enable trace comparisons and provide profiling results. However, none of these tools enforce a strict communication order during replay, limiting their ability to facilitate precise, apple-to-apple comparisons and identify points of divergence in distributed executions.

Handling Loops. Many differential analysis approaches also consider loops, as loop iterations often have intuitive meanings assigned by developers. Ignoring loops in identifying divergence or convergence points can lead to less intuitive results for users. Tools like DiffTrace and [1, 20] handle loops but fail to account for "unnatural" loop [36] cases, which are common in programs. The method

introduced in [19] builds on [36] to identify divergence and convergence points that respect loop iterations. In our work, we apply CFG unrolling to facilitate online alignment to make decisions on enforcing communication order during replay.

Record and Replay (RnR). The record-and-replay (RnR) technique has been extensively studied for cyclic debugging. Our implementation is based on ReMPI [24], specifically designed for the MPI framework. Additionally, we implicitly leverage concepts from order replay [14] and logical time [15] to ensure correctness and reduce performance overhead in our implementation. ANACIN-X [4], rather than suppress the non-deterministic behavior, reports to users the differences in communication patterns (not the path differences as our research). We referred to many of the experiment data from this paper, such as MCB [24] and AMG2013 [2].

7 conclusion

In this paper, we addressed the challenge of comparing two runs of a parallel MPI application in the presence of two sources of non-determinism: input changes and variations in message exchange order. Unlike traditional record-and-replay methods, which assume identical inputs across runs and are unable to handle non-deterministic message exchanges effectively, our approach leverages selective replay to align execution paths dynamically. Our work advances the state of differential analysis for MPI applications by explicitly addressing the challenges posed by non-deterministic communication and input variations. These results highlight the potential of selective replay as a tool for improving the reproducibility and debuggability of parallel applications. Future work may explore the scalability of our approach for larger applications and its applicability to other parallel programming models beyond MPI. Our open-source implemementation is available via [7].

References

[1] Xavier Aguilar, Karl Fürlinger, and Erwin Laure. 2015. Automatic On-Line Detection of MPI Application Structure with Event Flow Graphs. In *Euro-Par 2015: Parallel Processing*, Jesper Larsson Träff, Sascha Hunold, and Francesco Versaci (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 70–81.

[2] LLNL ASC. 2013. AMG2013: Algebraic Multigrid Solver. https://asc.llnl.gov/codes/proxy-apps/amg2013. Accessed: 2024-06-21.

[3] Zhuowei Bao, Sarah Cohen-Boulakia, Susan B Davidson, Anat Eyal, and Sanjeev Khanna. 2009. Differencing provenance in scientific workflows. In *ICDE*.

[4] P. Bell, K. Suarez, D. Chapp, N. Tan, S. Bhowmick, and M. Taufer. 2021. ANACIN-X: A Software Framework for Studying Non-Determinism in MPI Applications. *Software Impacts* 10 (2021), 100151. doi:10.1016/j.simpa.2021.100151

[5] H. Brunst and M. Weber. 2012. Custom Hot Spot Analysis of HPC Software with the Vampir Performance Tool Suite. In *Proc. Int'l Parallel Tools Wksp.* 95–114.

[6] Banghao Chen, Zhaofeng Zhang, Nicolas Langrené, and Shengxin Zhu. 2024. Unleashing the potential of prompt engineering in Large Language Models: a comprehensive review. arXiv:2310.14735 [cs.CL] https://arxiv.org/abs/2310.14735

[7] depaul dice. 2025. ProvScopeMPI: Extension of ProvScope in MPI Applications. https://github.com/depaul-dice/ProvScopeMPI/tree/easyGlobalAlignment Accessed: 2025-05-04.

[8] W. U. Hassan, M. Lemay, N. Aguse, A. Bates, and T. Moyer. 2018. Towards Scalable Cluster Auditing through Grammatical Inference over Provenance Graphs. In *NDSS*. https://api.semanticscholar.org/CorpusID:28601100

[9] Intel Corporation 2015. *Intel Trace Analyzer and Collector*. Intel Corporation. https://www.intel.com/content/www/us/en/developer/tools/oneapi/trace-analyzer.html

[10] Paul Jaccard. 1901. Étude comparative de la distribution florale dans une portion des Alpes et du Jura. *Bulletin de la Société Vaudoise des Sciences Naturelles* 37 (1901), 547–579.

[11] Y. Kwon, F. Wang, W. Wang, K. H. Lee, W. Lee, S. Ma, X. Zhang, D. Xu, S. Jha, G. F. Ciocarlie, A. Gehani, and V. Yegneswaran. 2018. MCI: Modeling-Based Causality Inference in Audit Logging for Attack Investigation. In *NDSS*.

[12] I. Laguna, K. Mohror, M. Ruefenacht, R. Marshall, A. Skjellum, and N. Sultana. 2019. A Large-Scale Study of MPI Usage in Open-Source HPC Applications. In *SC '19*. ACM, 1–14. doi:10.1145/3295500.3356176

[13] Chris Lattner. 2014. The architecture of open source applications: LLVM. *The architecture of open source applications* (2014).

[14] Leblanc and Mellor-Crummey. 1987. Debugging Parallel Programs with Instant Replay. *IEEE Trans. Comput.* C-36, 4 (1987), 471–482. doi:10.1109/TC.1987.1676929

[15] L. J. Levrouw, K. M. R. Audenaert, and J. M. Van Campenhout. 1994. A New Trace and Replay System for Shared Memory Programs Based on Lamport Clocks. In *Proc. 2nd Euromicro Workshop on Parallel and Distributed Processing*. 471–478. doi:10.1109/EMPDP.1994.592529

[16] Victor A. Magri, Robert D. Falgout, and Ulrike M. Yang. 2023. A New Semistructured Algebraic Multigrid Method. *SIAM Journal on Scientific Computing* 45, 3 (2023), S439–S460.

[17] M. Maruyama, T. Tsumura, and H. Nakashima. 2005. Parallel Program Debugging Based on Data-Replay. In *Proceedings of the IASTED International Conference on Parallel and Distributed Computing and Systems (PDCS'05)*. IASTED.

[18] H. Meng, R. Kommineni, Q. Pham, R. Gardner, T. Malik, and D. Thain. 2015. An Invariant Framework for Conducting Reproducible Computational Science. In *International Conference on Computational Science (ICCS)*. 137–142. doi:10.1016/j.jocs.2015.04.012 Acceptance Rate: 30%.

[19] Yuta Nakamura, Iyad Kanj, and Tanu Malik. 2023. Efficient Differencing of System-level Provenance Graphs. In *Proceedings of the 2023 ACM International Conference on Information and Knowledge Management (CIKM)*.

[20] Y. Nakamura, T. Malik, and A. Gehani. 2020. Efficient Provenance Alignment in Reproduced Executions. In *TaPP*. USENIX Association. https://www.usenix.org/conference/tapp2020/presentation/nakamura

[21] Yuta Nakamura, Tanu Malik, Iyad Kanj, and Ashish Gehani. 2022. Provenance-based Workflow Diagnostics Using Program Specification. In *Intl Conf. on High-Performance Computing, Data, and Analytics*. https://dice.cs.depaul.edu/pdfs/pubs/C32.pdf

[22] Harish Patil, Cristiano Pereira, Mack Stallcup, Gregory Lueck, and James Cownie. 2010. PinPlay: A Framework for Deterministic Replay and Reproducible Analysis of Parallel Programs. In *Proc. of Intl. Symposium on Code Generation and Optimization (CGO '10)*. ACM, 2–11.

[23] R. Ricci, E. Eide, G. Wong, L. Stoller, M. Hibler, J. Duerig, T. Stack, and K. Webb. 2016. Apt: A Platform for Repeatable Research in Computer Science. In *HPDC*. ACM, 331–334. doi:10.1145/2907294.2907301

[24] K. Sato, D. H. Ahn, I. Laguna, G. L. Lee, and M. Schulz. 2015. Clock Delta Compression for Scalable Order-Replay of Non-Deterministic Parallel Applications. In *SC '15*. ACM, 62:1–62:12. doi:10.1145/2807591.2807642

[25] Victoria Stodden and Matthew S. Krafczyk. 2018. Assessing Reproducibility: An Astrophysical Example of Computational Uncertainty in the HPC Context. https://stodden.net/papers/ResCuE2018-VSMK.pdf

[26] V. Stodden, M. McNutt, D. H. Bailey, E. Deelman, Y. Gil, B. Hanson, M. A. Heroux, J. P. A. Ioannidis, and M. Taufer. 2016. Enhancing Reproducibility for Computational Methods. *Science* 354, 6317 (2016), 1240–1241. doi:10.1126/science.aah6168

[27] Saeed Taheri et al. 2019. ParLOT: Efficient Whole-Program Call Tracing for HPC Applications. In *Programming and Performance Visualization Tools*. Vol. 6. Springer International Publishing, 162–184.

[28] Saeed Taheri, Ian Briggs, Martin Burtscher, and Ganesh Gopalakrishnan. 2019. DiffTrace: Efficient Whole-Program Trace Analysis and Diffing for Debugging. In *CLUSTER*. 1–12. doi:10.1109/CLUSTER.2019.8891027

[29] Dai Hai Ton That, Gabriel Fils, Zhihao Yuan, and Tanu Malik. 2017. Sciunits: Reusable Research Objects. In *IEEE eScience*.

[30] M. J. Turk, T. Abel, and B. O'Shea. 2009. Science. *Science* 325 (2009), 601.

[31] Jan Vitek and Tomas Kalibera. 2011. Repeatability, reproducibility and rigor in systems research. In *EMSOFT*. ACM.

[32] Robert A. Wagner and Michael J. Fischer. 1974. The String-to-String Correction Problem. *J. ACM* 21, 1 (jan 1974), 168–173. doi:10.1145/321796.321811

[33] G. H. Weber et al. 2010. Numerical Modeling of Space Plasma Flows: Astronum-2009. In *Astronomical Society of the Pacific Conference Series*, Vol. LBNL-3185E.

[34] M. Weber, R. Brendel, T. Hilbrich, K. Mohror, M. Schulz, and H. Brunst. 2016. Structural Clustering: A New Approach to Support Performance Analysis at Scale. In *IPDPS*. 484–493. doi:10.1109/IPDPS.2016.27

[35] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2023. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. https://arxiv.org/abs/2201.11903

[36] T. Wei, J. Mao, W. Zou, and Y. Chen. 2007. A New Algorithm for Identifying Loops in Decompilation. In *SAS*. Springer, 170–183.

[37] Rudolf Wille. 2009. Restructuring Lattice Theory: An Approach Based on Hierarchies of Concepts. In *Proc. of Intl. Conference on Formal Concept Analysis (ICFCA '09)*. Springer-Verlag, Berlin, Heidelberg, 314–339.

[38] R. Xue, X. Liu, M. Wu, Z. Guo, W. Chen, W. Zheng, Z. Zhang, and G. Voelker. 2009. MPIWiz: Subgroup Reproducible Replay of MPI Applications. In *Proc. of ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*.